

LABORATORY OBSERVATION

Course	Full Stack Web Development
Course Code	23CM4121
Experiment No.	5
Student Name	Kavya G.
Roll No.	A24126552082
Date	21-09-2026

1. TITLE

Develop a Student Server Using Express.js

2. AIM

To develop a basic **Express.js** server with routes `/`, `/students`, and `/about` to manage and display student information using HTTP methods, routing, and response handling.

3. OBJECTIVE

- To understand the basics of **Express.js**.
 - To create and run an Express.js server.
 - To implement different routes using `app.get()`.
 - To return student data in **JSON format**.
 - To understand HTTP methods, routing, and response handling.
-

4. THEORY

Express.js is a lightweight and flexible web application framework for **Node.js**. It is commonly used to create web servers and APIs.

An Express application can define different **routes** to handle requests from clients. A route consists of a URL path and an HTTP method such as GET, POST, PUT, or DELETE.

In this experiment, the GET method is used to retrieve information from the server.

The `app.get()` method is used to define GET routes:

```
app.get('/path', (req, res) => {  
  
  // response  
  
});
```

Here:

- `req` represents the request received from the client.
- `res` represents the **response** sent by the server.

- `res.send()` sends a text response.
 - `res.json()` sends data in JSON format
-

5. ALGORITHM / PROCEDURE

1. Create a new Node.js project.
 2. Install the Express.js package using npm.
 3. Import the Express module.
 4. Create an Express application.
 5. Define the server port as 3000.
 6. Create an array containing student information.
 7. Create the `/` route using `app.get()`.
 8. Create the `/students` route and return the student data using `res.json()`.
 9. Create the `/about` route and return application information using `res.send()`
 10. Start the server using `app.listen()`.
 11. Open the routes in a web browser and verify the outputs.
-

6. PROGRAM

```
const express = require('express');
const app = express();
const port = 3000;
// Student data
const students = [
  {
    id: 1,
    name: "Rahul",
    course: "CSE"
  },
  {
    id: 2,
    name: "Priya",
    course: "AI & ML"
  },
  {
    id: 3,
    name: "Arjun",
    course: "ECE"
  },
  {
    id: 4,
    name: "Sneha",
    course: "CSE"
  },
  {
    id: 5,
    name: "Kiran",
    course: "IT"
  }
];
// Home route
app.get('/', (req, res) => {
  res.send('Welcome to Student Server!');
```

```

    });
    // Home route
    app.get('/', (req, res) => {
        res.send('Welcome to Student Server!');
    });
    // Students route
    app.get('/students', (req, res) => {
        res.json(students);
    });
    // About route
    app.get('/about', (req, res) => {
        res.send('This application manages student information using Express.js.');
```

```

    });
    // Start server
    app.listen(port, () => {
        console.log(`Server is running on port ${port}`);
    });

```

7. CODE EXPLANATION

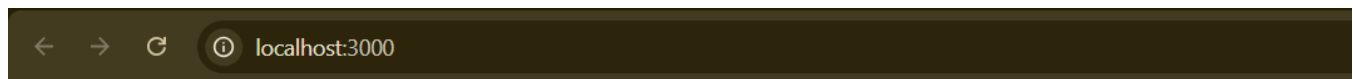
Code / Function	Explanation
require('express')	imports Express.js.
express()	Creates the Express app.
port = 3000	Sets the server port.
students = [...]	Stores student details.
app.get('/')	Defines the home route.
app.get('/students')	Defines the students route.
res.json(students)	Sends student data as JSON.
app.get('/about')	Defines the about route.
res.send()	Sends a text response.
app.listen()	Starts the server.
console.log()	Displays a message in the terminal.

8. TEST CASES AND EXECUTION DATA

Test Case	URL	Expected Result
TC-01	http://localhost:3000/	Welcome to Student Server!
TC-02	http://localhost:3000/students	List of five students in JSON format
TC-03	http://localhost:3000/about	Application information

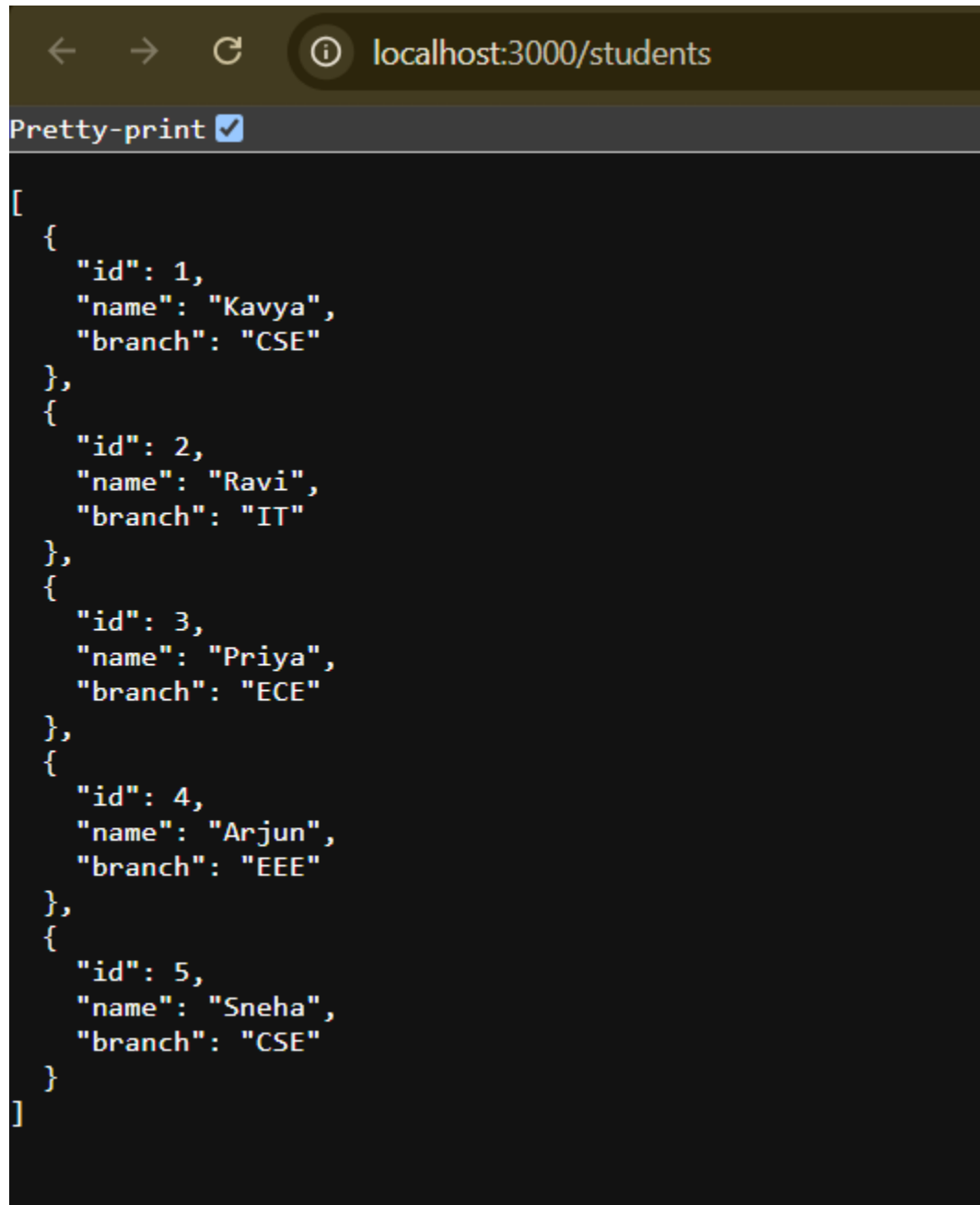
9. OUTPUT

TC-01 : Home Route



Welcome to Student Server!

TC-02 : Students Route



```
[
  {
    "id": 1,
    "name": "Kavya",
    "branch": "CSE"
  },
  {
    "id": 2,
    "name": "Ravi",
    "branch": "IT"
  },
  {
    "id": 3,
    "name": "Priya",
    "branch": "ECE"
  },
  {
    "id": 4,
    "name": "Arjun",
    "branch": "EEE"
  },
  {
    "id": 5,
    "name": "Sneha",
    "branch": "CSE"
  }
]
```

TC-03 : : About Route



10. RESULT

Thus, a basic **Express.js Student Server** was successfully developed with /, /students, and /about routes. The application successfully demonstrates **Express.js setup, server creation, routing, HTTP GET methods, and response handling**.